

1.2 Why Vector Calculus

Training is an optimization problem. Linear regression is the first example.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

The figures follow Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*.

1. Training is optimization

Many algorithms in this course **optimize an objective** with respect to model parameters θ that control how well the model explains the data. Finding good parameters is an optimization problem. Three pictures you will see again:

- **Linear regression.** θ are weights. Maximize a likelihood, or minimize squared error.
- **Auto-encoders** (Module 6). θ are layer weights and biases. Minimize reconstruction error by repeated application of the chain rule.
- **Gaussian mixtures** (Module 5). θ are means, shapes, and mixture weights. Maximize the likelihood. Used in clustering and anomaly detection.

We write the objective as $J(\theta): \mathbb{R}^l \rightarrow \mathbb{R}$. Training is

$$\hat{\theta} \in \arg \min_{\theta} J(\theta).$$

If you cannot write J and say what θ is, you do not yet have a learning problem.

2. Linear regression, written so the calculus is visible

Training pairs (y_n, x_n) , $n = 1, \dots, N$. The prediction model is

$$\hat{y}_n = \hat{\theta}_0 + \hat{\theta}_1 x_1 + \dots + \hat{\theta}_l x_l = \hat{\theta}^\top x_n.$$

Estimate θ by minimizing squared error

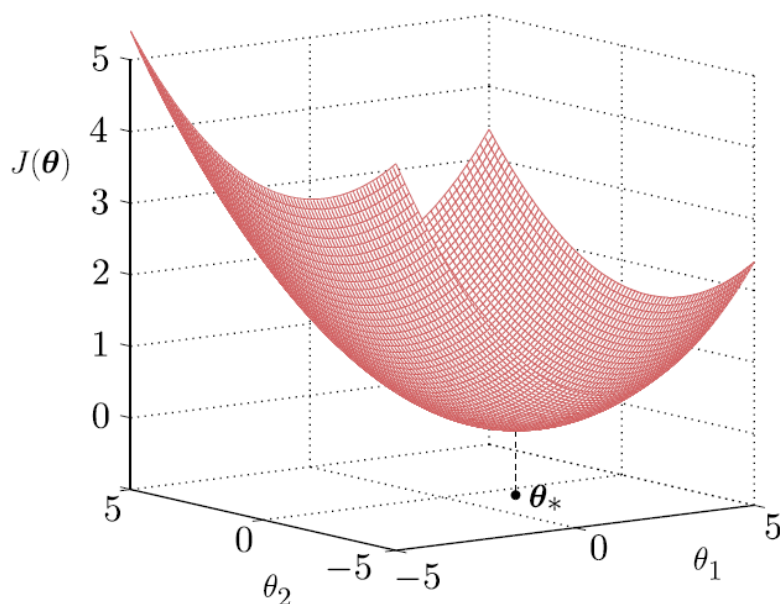
$$J(\theta) = \sum_{n=1}^N (y_n - \theta^\top x_n)^2.$$

To minimize J , solve $\partial J / \partial \theta = 0$. In matrix form that is the **normal equation**

$$(\mathbf{X}^\top \mathbf{X}) \hat{\theta} = \mathbf{X}^\top \mathbf{y}, \quad \hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

when $\mathbf{X}^\top \mathbf{X}$ is invertible. The inverse existing is a linear-algebra fact (full column rank). The $\nabla J = 0$ step is calculus.

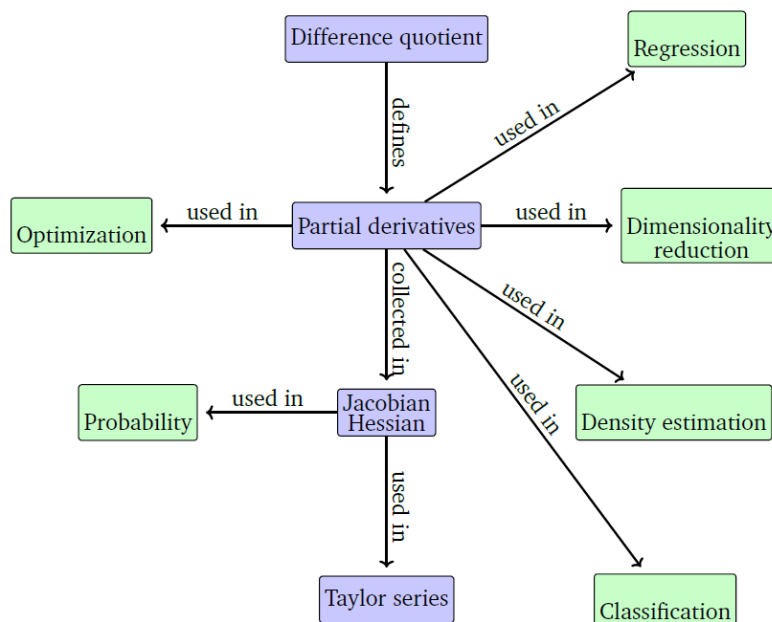
For two weights the surface J is a convex bowl. One local minimum, and it is global.



Neural nets will not be this kind.

3. A map of the math

The next figure is the course's reminder of which math shows up where. Module 1 lives in the vector-calculus and optimization blocks. Later modules reuse the same objects (gradients, Hessians, inner products, eigenvalues) on new models.



To minimize a smooth J you need:

- a **gradient** (first derivatives) — when to stop, which way is downhill;
- a **Hessian** (second derivatives) — how curved the bowl is;
- later, **constraints** (Week 3) when θ is not free in $\mathbb{R}^l$.

The next two notes are those derivatives. Week 2 turns them into algorithms.

4. Practice

1. Why is the normal equation a calculus problem, not a programming problem?
2. In the auto-encoder example, what is J , and why does the chain rule appear?
3. If $\mathbf{X}^T \mathbf{X}$ is not invertible, what did the data do, and what do you change before you invert?